

بخش ۱

سؤال ۲) چرا از JOIN استفاده شده؟ تفاوت با LEFT JOIN

چرا JOIN ؟

JOIN فقط رکوردهایی را می‌آورد که در هر دو جدول مرتبط اطلاعات معتبر دارند.

در اینجا:

هر تعمیر باید یک مشتری داشته باشد -> UserID

هر تعمیر باید یک دستگاه داشته باشد -> DeviceID

هر دستگاه باید یک مدل معتبر داشته باشد -> ModelID

پس اگر رکوردی در هر کدام ناقص باشد، منطقی نیست در داشبورد نمایش داده شود.

اگر LEFT JOIN استفاده شود چه اتفاقی می‌افتد؟

INNER JOIN فقط تعمیرات با اطلاعات کامل مشتری/دستگاه/مدل نمایش داده می‌شود.

LEFT JOIN همه تعمیرات نمایش داده می‌شوند حتی اگر اطلاعات مشتری یا مدل ناقص باشد. ستون‌های از دست‌رفته مقدار NULL می‌گیرند.

مثال:

اگر کارشناس قبلی یک دستگاه را ثبت کرده باشد ولی مدل را اشتباه پر کرده باشد:

با INNER JOIN → این تعمیر در خروجی نمی‌آید.

با LEFT JOIN → تعمیر نشان داده می‌شود ولی ModelName برابر NULL می‌شود.

چرا اینجا LEFT JOIN مناسب نیست؟

ممکن است نام مشتری NULL باشد -> نمی‌دانیم دستگاه مال چه کسی است

مدل NULL باشد -> اطلاعات گزارش مالی و آماری خراب می‌شود

سؤال ۳ (نقش شرط $AdminID = ?$ چیست؟)

این شرط یک فیلتر امنیتی و کنترلی است.

WHERE r.AdminID = ? باعث می‌شود فقط تعمیراتی نمایش داده شوند که توسط ادمین مشخص (یا کارمند خاص) ثبت شده‌اند.

به عبارت ساده:

این کوئری فقط رکوردهای تعمیرات مربوط به مدیر/کارشناس مشخصی را نشان می‌دهد.

در چه سناریوهایی ضروری است؟

۱. نقش‌بندی و دسترسی (RBAC)

هر ادمین یا کارمند فقط باید تعمیرات خودش را ببیند:

کارشناس الف => فقط تعمیرات خودش

کارشناس ب => فقط تعمیرات خودش

بدون این شرط، یک کارمند می‌توانست رکوردهای دیگران را هم ببیند => نقص امنیتی.

۲. استفاده در داشبورد شخصی

اگر داشبورد کارمند نمایش "تعمیرات امروز من" باشد، باید بر اساس AdminID فیلتر شود.

۳. ممیزی (Audit)

اگر مدیر بخواهد ببیند چه تعمیراتی توسط چه کسی ثبت شده این فیلتر ضروری می‌شود.

۴ (نقش ORDER BY r.DateIn DESC)

✓ این دستور چه می‌کند؟

ORDER BY r.DateIn DESC

یعنی رکورد‌های تعمیرات را بر اساس تاریخ ورود دستگاه مرتب می‌کند، از جدید به قدیم.

(تازمترین تعمیرات اول نمایش داده می‌شوند، قدیمی‌ها پایین‌تر)

چرا نزولی؟

چون در داشبورد تعمیرات معمولاً کاربر دنبال کارهای امروز و تازمترین دستگاه‌های رسیده است.

اگر تاریخ به‌صورت صعودی (ASC) مرتب شود:

قدیمی‌ترین تعمیرات (مثلاً مال یک سال پیش) اول نمایش داده می‌شود

تعمیرکار باید اسکرویل زیادی کند → افت کارایی

یک دلیل عملی انتخاب این ترتیب

مثال واقعی در تعمیرگاه موبایل:

کارشناس وارد داشبورد می‌شود تا ببیند:

دستگاه‌هایی که امروز یا دیروز وارد شده‌اند

کارهایی که در صف هستند

موارد فوری یا VIP

اگر آخرین تعمیرات اول نشان داده شوند:

وضعیت تعمیرات بهروز را سریع می بیند

می تواند کارهای جدید را فوراً پیگیری کند

از مهلت های تحویل عقب نمی افتد

بخش ۲

سؤال (۱) این کوئری چه چیزی را می خواهد نشان دهد؟

این کوئری ساختار «لیست قیمت تعمیرات موبایل» را برای هر دسته موبایل (Category) به صورت کامل نشان می دهد.

در اصل، کوئری تلاش می کند نمایش دهد که:

در هر PhoneCategory چه PriceList هایی وجود دارد

در هر PriceList چه Service هایی تعریف شده است

هر Service برای چه Model هایی چه قیمتی دارد

این کوئری یک ساختار چندبعدی می سازد:

Category → PriceList → Services → Models → PriceValue

سؤال (۲) چرا از چند LEFT JOIN استفاده شده است؟

دلیل ۱ :

داده های ناقص باید همچنان نمایش داده شوند

در سیستم قیمت گذاری:

ممکن است Category وجود داشته باشد ولی PriceList نداشته باشد

ممکن است PriceList وجود داشته باشد ولی Service تعریف نشده باشد

ممکن است Model باشد ولی قیمت آن ثبت نشده باشد

اگر INNER JOIN استفاده می‌شد، دسته‌هایی که هنوز برایشان لیست قیمت تعریف نشده بود کاملاً از خروجی حذف می‌شدند.

LEFT JOIN تضمین می‌کند:

«همه دسته‌ها نشان داده شوند، حتی اگر زیرمجموعه‌هایشان تکمیل نشده باشد».

دلیل ۲:

برای ساخت فرم/داشبورد قیمت‌گذاری نیاز است همه داده‌ها دیده شوند

در داشبورد مدیریت قیمت:

ادمین باید همه دسته‌ها را ببیند

حتی اگر برای آن‌ها هنوز PriceList ساخته نشده باشد

یا قیمت برخی مدل‌ها هنوز مقدار نداشته باشد

LEFT JOIN مانع از حذف ردیف‌ها می‌شود.

دلیل ۳:

رابطه‌ها تو در تو (Nested) هستند

این زنجیره کاملاً به هم وابسته است:

Category → PriceList → Services → PriceValues → Models

اگر هر بخش NULL باشد، بخش‌های بعدی هم NULL می‌شوند اما **Category** همچنان حفظ می‌شود.

سؤال ۴

در جدول (pv PriceValues) هر قیمت وابسته به سه چیز است:

1. PriceListID → قیمت مربوط به کدام لیست قیمت/ادمین است
2. ModelID → برای کدام مدل گوشی است
3. ServiceID → قیمت کدام خدمت است

چون یک سرویس برای مدل‌های مختلف قیمت متفاوت دارد و یک مدل برای سرویس‌های مختلف قیمت متفاوت دارد و هر ادمین لیست قیمت جدا دارد، پس قیمت فقط با ترکیب سه‌گانه زیر تعریف می‌شود:

(PriceListID, ModelID, ServiceID)

اگر یکی حذف شود، قیمت مبهم یا اشتباه می‌شود.

بخش ج)

```
var query = "SELECT * FROM Users WHERE id = " + input;
```

مشکل اصلی :

SQL Injection

چون ورودی کاربر مستقیم داخل کوئری قرار می‌گیرد، کاربر می‌تواند ورودی خطرناک بدهد مثل:

OR 1=1 1

این باعث می‌شود:

کل داده‌ها لو برود

کوئری خراب یا مخرب اجرا شود

امنیت سیستم کاملاً از بین برود

راه درست (Parameterization)

کوئری باید به صورت پارامتری نوشته شود:

```
db.query("SELECT * FROM Users WHERE id = ?", [input[0];
```

با این روش، ورودی تبدیل به داده می‌شود نه کد؛

بنابر این حمله‌ی SQL Injection غیر ممکن می‌شود.

سوال ۲)

بخش ۱ :

مدل‌سازی منطقی (Logical Model)

موجودیت‌ها (Entities)

1. Farm

farm_id (PK)

farm_name

location

2. Crop (هر مزرعه چند محصول دارد)

crop_id (PK)

farm_id (FK → Farm)

crop_name

planting_date

3. Activity (نوع فعالیت‌ها: آبیاری، کوددهی، برداشت...)

activity_id (PK)

activity_name

description

4. Employee (کارگران)

employee_id (PK)

full_name

phone

5. (Work_Records رابطه چند به چند بین Employee و Activity و Crop)

record_id (PK)

employee_id (FK → Employee)

crop_id (FK → Crop)

activity_id (FK → Activity)

hours_worked

payment

date

روابط (Relationships)

هر Farm → چند Crop دارد (N:1)

هر Crop → چند Activity روی آن انجام می‌شود (N:M)

هر Activity می‌تواند توسط چند Employee انجام شود (N:M)

هر Employee روی چند Crop کار می‌کند → از طریق Work_Records

بنابراین جدول Work_Records رابطه چندبه‌چند را پوشش می‌دهد و اطلاعات عملیاتی (ساعات، مبلغ) را نیز نگه می‌دارد.

بخش دوم

دستورات:

```
CREATE TABLE Farm (
```

```
    farm_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    farm_name VARCHAR(100),
```

```
    location VARCHAR(200)
```

```
);
```



```
CREATE TABLE Crop (  
    crop_id INT PRIMARY KEY AUTO_INCREMENT,  
    farm_id INT,  
    crop_name VARCHAR(100),  
    planting_date DATE,  
    FOREIGN KEY (farm_id) REFERENCES Farm(farm_id)  
);
```

```
CREATE TABLE Activity (  
    activity_id INT PRIMARY KEY AUTO_INCREMENT,  
    activity_name VARCHAR(100),  
    description TEXT  
);
```

```
CREATE TABLE Employee (  
    employee_id INT PRIMARY KEY AUTO_INCREMENT,  
    full_name VARCHAR(100),  
    phone VARCHAR(20)  
);
```

```
CREATE TABLE Work_Records (  
    record_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
employee_id INT,  
crop_id INT,  
activity_id INT,  
hours_worked DECIMAL(5,2),  
payment DECIMAL(10,2),  
date DATE,  
FOREIGN KEY (employee_id) REFERENCES Employee(employee_id),  
FOREIGN KEY (crop_id) REFERENCES Crop(crop_id),  
FOREIGN KEY (activity_id) REFERENCES Activity(activity_id));
```